

QUESTION NO: 20

- Write pseudocode for three periodic software timers activating different tasks.

PROBLEM UNDERSTANDING:

Three software timers with different periods are required to periodically activate three different tasks in an RTOS. On expiry of each timer, the corresponding task is triggered using events.

RTOS MECHANISM USED:

Software Timers and Event Flags

STRUCTURED PSEUDOCODE:

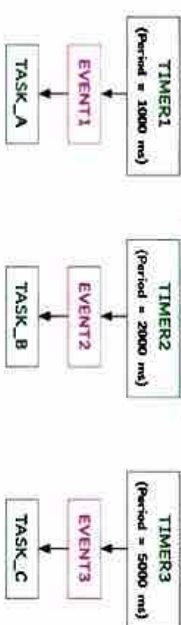
// GLOBAL DEFINITIONS

- DEFINE TIMER Timer1, Timer2, Timer3
- DEFINE EVENT Event1, Event2, Event3
- DEFINE TASK Task_A, Task_B, Task_C

DATA STRUCTURE: Timer Control Block (TCB)

```
TYPE Timer
    period      : INTEGER
    next_expiry : TIME
    event       : EVENT
ENDTYPE
```

SYSTEM OVERVIEW



1

PSEUDOCODE (Contd...)

BEGIN

// SYSTEM INITIALIZATION

```

CREATE TIMER Timer1 WITH PERIOD 1000
CREATE TIMER Timer2 WITH PERIOD 2000
CREATE TIMER Timer3 WITH PERIOD 5000
CREATE EVENT Event1
CREATE EVENT Event2
CREATE EVENT Event3
CREATE TASK Task_A WITH PRIORITY 2
CREATE TASK Task_B WITH PRIORITY 2
CREATE TASK Task_C WITH PRIORITY 2
START SCHEDULER
  
```

END

TIMER SERVICE TASK

BEGIN

LOOP FOREVER

CurrentTime ← GET_TIME()

// Check Timer1

IF CurrentTime ≥ Timer1.next_expiry THEN

SET_EVENT(Event1)

Timer1.next_expiry ← CurrentTime + Timer1.period

ENDIF

// Check Timer2

IF CurrentTime ≥ Timer2.next_expiry THEN

SET_EVENT(Event2)

Timer2.next_expiry ← CurrentTime + Timer2.period

ENDIF

// Check Timer3

IF CurrentTime ≥ Timer3.next_expiry THEN

SET_EVENT(Event3)

Timer3.next_expiry ← CurrentTime + Timer3.period

ENDIF

WAIT(10) // small delay

ENDLOOP

END

2

PSEUDOCODE (Contd...)

TASK: Task_A

BEGIN

LOOP FOREVER

WAIT_EVENT(Event1)

PERFORM Task_A_Activity()

ENDLOOP

END

TASK: Task_B

BEGIN

LOOP FOREVER

WAIT_EVENT(Event2)

PERFORM Task_B_Activity()

ENDLOOP

END

TASK: Task_C

BEGIN

LOOP FOREVER

WAIT_EVENT(Event3)

PERFORM Task_C_Activity()

ENDLOOP

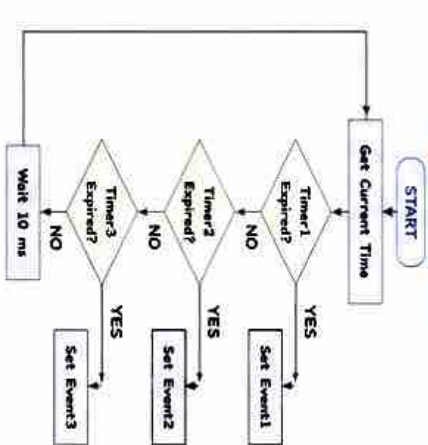
END

QUEUE OPERATIONS SUMMARY

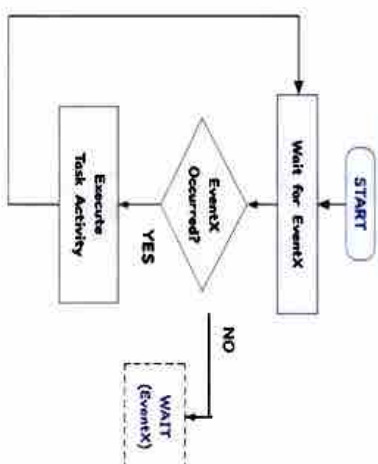
Operation	Performed By	Purpose
SET_EVENT(EventX)	Timer Service Task	Signals the event for corresponding task
WAIT_EVENT(EventX)	Task_X	Waits for event before execution
GET_TIME()	Timer Service Task	Provides current system time

3

TIMER SERVICE TASK



TASK X (A/B/C)



4

EXPLANATION:

Three software timers are created with different periods (1000ms, 2000ms, 5000ms). A Timer Service Task checks the timers periodically.

When a timer expires, it sets an event for the corresponding task. Each task waits for its event and executes its activity when activated. This ensures periodic activation of different tasks using software timers.

KEY POINTS:

- Software timers generate events periodically.
- Events are used to activate tasks.
- Each timer is independent and has its own period.
- Tasks block (wait) until their event is set.

ASSUMPTIONS:

- Timer service task runs at regular intervals (e.g., every 10 ms).
- GET_TIME() returns system time in ms.
- Tasks are non-blocking except when waiting for events.

5

QUESTION NO: 21

- Write pseudocode for a software watchdog monitoring a critical task.

PROBLEM UNDERSTANDING:

A software watchdog monitors a critical task by periodically expecting a "kick" (feed) from it. If the kick is not received within the timeout period, the watchdog triggers a recovery action.

RTOS MECHANISM USED:

Periodic Timer, Shared Flag/Counter, Task Monitoring

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

```
- DEFINE WATCHDOG_TIMEOUT    2000    // in ms
- DEFINE WATCHDOG_PERIOD     200     // in ms
- DEFINE KICK_FLAG            BOOLEAN
```

// GLOBAL VARIABLES

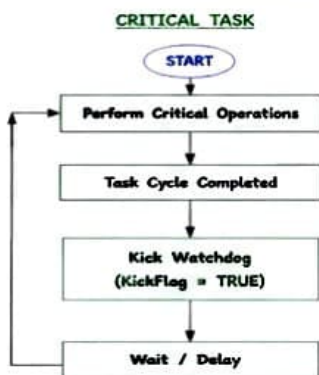
WatchdogCounter : INTEGER

KickFlag : BOOLEAN

WATCHDOG STATES

- NORMAL** -> Kick received in time
- TIMEOUT** -> No kick within timeout
- RECOVERY** -> Recovery action in progress

SYSTEM OVERVIEW



PSEUDOCODE (Contd...)

TASK: CriticalTask (The task being monitored)

BEGIN

LOOP FOREVER

... // Perform critical operations

... // On successful completion

KickFlag ← TRUE // Kick the watchdog

ENDLOOP

END

TASK: WatchdogTask (The watchdog monitor)

BEGIN

WatchdogCounter ← 0

KickFlag ← FALSE

LOOP FOREVER

WAIT FOR WATCHDOG_PERIOD // Periodic check

IF KickFlag = TRUE THEN

KickFlag ← FALSE

WatchdogCounter ← 0 // Reset counter

ELSE

WatchdogCounter ← WatchdogCounter + WATCHDOG_PERIOD

ENDIF

// Check for timeout

IF WatchdogCounter ≥ WATCHDOG_TIMEOUT THEN

TRIGGER_RECOVERY() // Call recovery handler

WatchdogCounter ← 0 // Reset after recovery

ENDIF

ENDLOOP

END

1

PSEUDOCODE (Contd...)

PROCEDURE: TRIGGER_RECOVERY()

BEGIN

// Enter recovery state

DISABLE_INTERRUPTS()

LOG_EVENT("Watchdog Timeout! Taking recovery action.")

SET_SYSTEM_TO_SAFE_STATE() // e.g., stop actuators

NOTIFY_SUPERVISOR() // LEDs / buzzer / message

SOFTWARE_RESET() // or system restart

END

WATCHDOG OPERATIONS SUMMARY

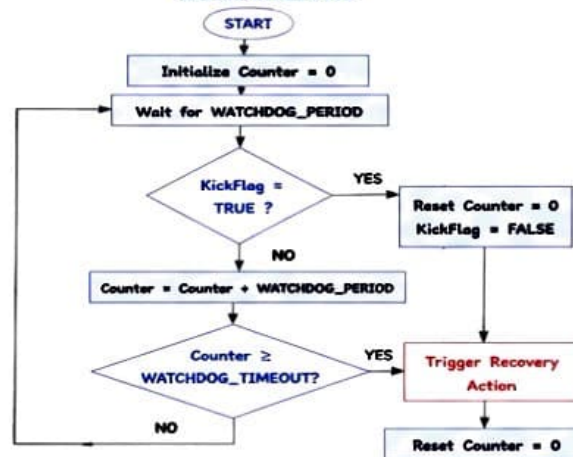
Operation	Performed By	Purpose
Kick Watchdog	Critical Task	Signals task is alive
Monitor Timeout	Watchdog Task	Checks if kick is received within timeout
Trigger Recovery	Watchdog Task	If timeout occurs, take recovery action

KEY POINTS:

- Watchdog must be kicked (fed) periodically by the critical task.
- Watchdog task resets the counter on each kick.
- If no kick is received within WATCHDOG_TIMEOUT, recovery action is triggered.
- This improves system reliability and prevents system lockups.

2

WATCHDOG TASK



3

EXPLANATION:

The Critical Task must kick the watchdog periodically after completing its work. The Watchdog Task monitors the kicks. If it does not receive a kick within the timeout period, it assumes the critical task is stuck or failed and triggers a recovery action to bring the system to a safe state.

ASSUMPTIONS:

- Both Critical Task and Watchdog Task run in an RTOS.
- WATCHDOG_PERIOD should be much smaller than WATCHDOG_TIMEOUT.
- Recovery action may be system reset or safe shutdown based on application.

STATE DIAGRAM



4

QUESTION NO: 22

- Write pseudocode where one controller task wakes multiple waiting tasks using event flags.

PROBLEM UNDERSTANDING:

A controller task sets event flags to signal multiple events. Multiple tasks wait on different event flags and get unblocked when their corresponding event flag is set.

RTOS MECHANISM USED:

Event Flags, Task Synchronization

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

- DEFINE EVENT_FLAG1 0x01 // Event for Task_A
- DEFINE EVENT_FLAG2 0x02 // Event for Task_B
- DEFINE EVENT_FLAG3 0x04 // Event for Task_C
- CREATE EVENT_FLAG_GROUP EventGroup

SYSTEM OVERVIEW



PSEUDOCODE (Contd...)

// CONTROLLER TASK

TASK: ControllerTask

BEGIN

LOOP FOREVER

// Simulate events and set flags

SET_EVENT_FLAG(EventGroup, EVENT_FLAG1)

WAIT(100) // small delay

SET_EVENT_FLAG(EventGroup, EVENT_FLAG2)

WAIT(100)

SET_EVENT_FLAG(EventGroup, EVENT_FLAG3)

WAIT(200)

ENDLOOP

END

// TASK A

TASK: Task_A

BEGIN

LOOP FOREVER

WAIT_FOR_EVENT_FLAG(EventGroup, EVENT_FLAG1)

PERFORM Task_A_Activity()

CLEAR_EVENT_FLAG(EventGroup, EVENT_FLAG1)

ENDLOOP

END

PSEUDOCODE (Contd...)

// TASK B

TASK: Task_B

BEGIN

LOOP FOREVER

WAIT_FOR_EVENT_FLAG(EventGroup, EVENT_FLAG2)

PERFORM Task_B_Activity()

CLEAR_EVENT_FLAG(EventGroup, EVENT_FLAG2)

ENDLOOP

END

// TASK C

TASK: Task_C

BEGIN

LOOP FOREVER

WAIT_FOR_EVENT_FLAG(EventGroup, EVENT_FLAG3)

PERFORM Task_C_Activity()

CLEAR_EVENT_FLAG(EventGroup, EVENT_FLAG3)

ENDLOOP

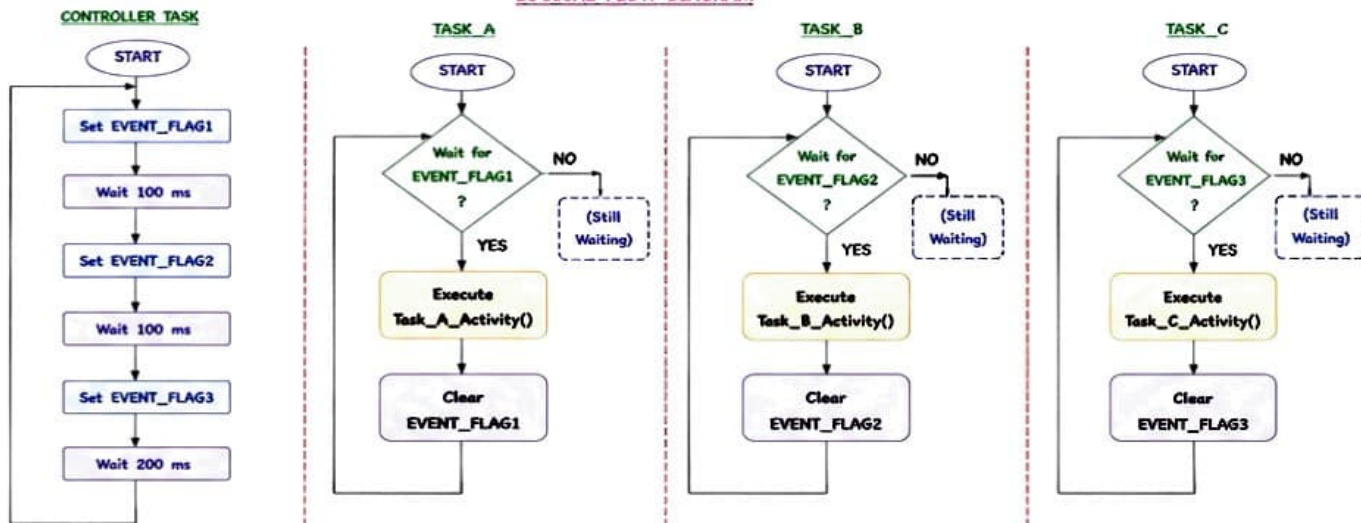
END

PSEUDOCODE (Contd...)

EVENT FLAG OPERATIONS SUMMARY

Operation	Performed By	Purpose
SET_EVENT_FLAG()	Controller Task	Signals event occurrence
WAIT_FOR_EVENT_FLAG()	Waiting Task	Blocks until the flag is set
CLEAR_EVENT_FLAG()	Waiting Task	Clears the flag after handling event

LOGICAL FLOW DIAGRAM



EXPLANATION:

Controller Task sets different event flags to signal events. Each task waits for its own event flag. When set, the task executes its activity and clears the flag.

KEY POINTS:

- Event flags allow one task to wake multiple waiting tasks.
- Tasks block (wait) until their event flag is set.
- Efficient for event-driven applications.

ASSUMPTIONS:

- All tasks and event flag group are created successfully.
- Event flags are independent and do not interfere.

QUESTION NO: 23

- Write pseudocode using message queues for communication between two tasks.

PROBLEM UNDERSTANDING:

Two tasks need to exchange data using a message queue. One task sends messages, and the other receives them. The message queue ensures safe communication and synchronization between tasks.

RTOS MECHANISM USED:

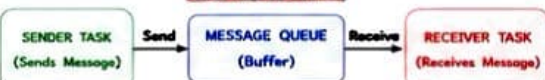
Message Queues, Task Synchronization

STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- DEFINE QUEUE_HANDLE    MsgQueue
- DEFINE MESSAGE_SIZE    32      // bytes
- DEFINE QUEUE_LENGTH    5       // messages
- DEFINE SEND_TASK       SenderTask
- DEFINE RECEIVE_TASK    ReceiverTask
```

```
// DATA TYPE
MessageType : ARRAY[0...MESSAGE_SIZE-1] OF CHAR
```

SYSTEM OVERVIEW



KEY QUEUE FUNCTIONS

- ◆ CREATE_QUEUE() - Create a message queue
- ◆ SEND() - Send message to queue
- ◆ RECEIVE() - Receive message from queue

PSEUDOCODE (Contd...)

// INITIALIZATION

BEGIN

```
CREATE_QUEUE(QUEUE_HANDLE, QUEUE_LENGTH,
             MESSAGE_SIZE) // Create queue
CREATE_TASK(SEND_TASK, SenderTask) // Task 1
CREATE_TASK(RECEIVE_TASK, ReceiverTask) // Task 2
START_SCHEDULER()
```

END

// SENDER TASK

TASK: SenderTask

BEGIN

```
DECLARE msg: MessageType
DECLARE count: INTEGER
count ← 1
LOOP FOREVER
    FORMAT(msg, "Message %d", count) // Prepare msg
    SEND(QUEUE_HANDLE, &msg, WAIT_FOREVER)
    // Send message to queue
    LOG_EVENT("Sent: " + msg)
    count ← count + 1
    DELAY(1000) // Wait 1 sec before next msg
```

ENDLOOP

END

1

PSEUDOCODE (Contd...)

// RECEIVER TASK

TASK: ReceiverTask

BEGIN

```
DECLARE msg: MessageType
```

LOOP FOREVER

```
RECEIVE(QUEUE_HANDLE, &msg, WAIT_FOREVER)
// Wait and receive message
LOG_EVENT("Received: " + msg)
PROCESS_MESSAGE(msg) // Application specific
```

ENDLOOP

END

// SUPPORT FUNCTION

FUNCTION: PROCESS_MESSAGE(msg: MessageType)

BEGIN

```
// Example: Print or handle message
PRINT("Processing -> " + msg)
// Add application logic here
```

END

2

PSEUDOCODE (Contd...)

MESSAGE QUEUE OPERATIONS SUMMARY

Operation	Performed By	Purpose
CREATE_QUEUE()	Initialization	Creates the message queue with size and length
SEND()	Sender Task	Sends message to the queue (blocks if full)
RECEIVE()	Receiver Task	Receives message from the queue (blocks if empty)
LOG_EVENT()	Both Tasks	Logs the send/receive activities

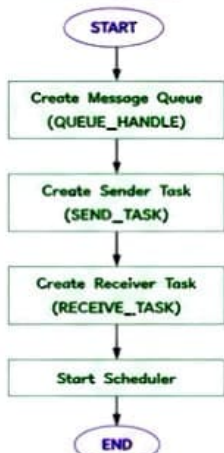
QUEUE PROPERTIES:

- ◆ FIFO (First In First Out) order is maintained.
- ◆ Ensures safe communication between tasks.
- ◆ Prevents data corruption and race conditions.
- ◆ Blocks sender if queue is full.
- ◆ Blocks receiver if queue is empty.

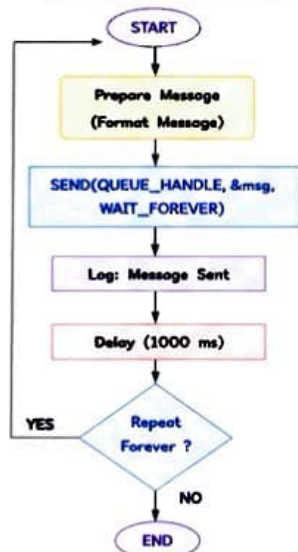
3

LOGICAL FLOW DIAGRAM

INITIALIZATION



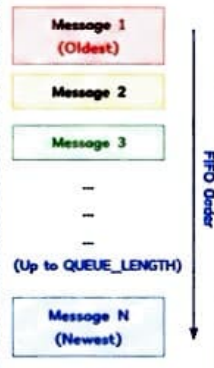
SENDER TASK (SenderTask)



RECEIVER TASK (ReceiverTask)



QUEUE BUFFER (Queue Content)



EXPLANATION:

The Sender Task creates and sends messages to the message queue. The Receiver Task waits for messages in the queue and processes them. Message queue ensures synchronized and reliable data exchange between tasks using FIFO order.

KEY POINTS:

- ◆ Message queue provides safe inter-task communication.
- ◆ Sender is blocked if queue is full.
- ◆ Receiver is blocked if queue is empty.
- ◆ FIFO order guarantees data integrity.
- ◆ Suitable for data exchange between tasks with different rates.

ASSUMPTIONS:

- Both tasks have same priority for simplicity.
- Queue length and message size are predefined.
- WAIT_FOREVER used for blocking operations.
- PROCESS_MESSAGE() is application specific.

4

QUESTION NO: 24

- Write pseudocode to implement message queue communication between a sensor task and a display task in an RTOS.

PROBLEM UNDERSTANDING:

A sensor task periodically reads data and sends it to a display task using a message queue. The display task receives the data from the queue and shows it.

RTOS MECHANISM USED:

Message Queue, Task Synchronization

STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- DEFINE MSG_QUEUE      MessageQueue
- DEFINE SENSOR_DATA    INTEGER
- DEFINE QUEUE_LENGTH   10    // No. of messages
```

GLOBAL VARIABLES

Message msg



PSEUDOCODE (Contd...)

// INITIALIZATION TASK (Executed once)

TASK: InitTask

BEGIN

```
CREATE_MESSAGE_QUEUE(MSG_QUEUE, QUEUE_LENGTH)
CREATE_TASK(SensorTask, PRIORITY_HIGH)
CREATE_TASK(DisplayTask, PRIORITY_MEDIUM)
START_TASK(SensorTask)
START_TASK(DisplayTask)
```

END

// SENSOR TASK (Producer)

TASK: SensorTask

BEGIN

LOOP FOREVER

// Read data from sensor

SENSOR_DATA = READ_SENSOR()

// Send data to queue (block if queue is full)

msg.data = SENSOR_DATA

SEND_MESSAGE(MSG_QUEUE, &msg, WAIT_FOREVER)

// Wait for next sampling period

WAIT(SAMPLING_PERIOD)

ENDLOOP

END

1

PSEUDOCODE (Contd...)

// DISPLAY TASK (Consumer)

TASK: DisplayTask

BEGIN

LOOP FOREVER

// Receive data from queue (block if queue is empty)

RECEIVE_MESSAGE(MSG_QUEUE, &msg, WAIT_FOREVER)

// Display the received data

SHOW_DATA_ON_DISPLAY(msg.data)

ENDLOOP

END

// FUNCTION: READ_SENSOR

FUNCTION: READ_SENSOR() : INTEGER

BEGIN

// Dummy function to read sensor value

value = GET_SENSOR_VALUE()

RETURN value

END

// FUNCTION: SHOW_DATA_ON_DISPLAY

FUNCTION: SHOW_DATA_ON_DISPLAY(data : INTEGER)

BEGIN

// Display the data on LCD/LED/Screen

DISPLAY(data)

END

2

MESSAGE QUEUE OPERATIONS SUMMARY

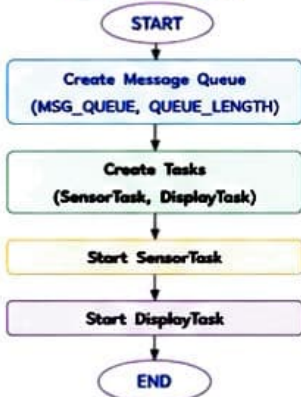
Operation	Performed By	Purpose
CREATE_MESSAGE_QUEUE()	Init Task	Creates message queue for communication
SEND_MESSAGE()	Sensor Task	Sends sensor data to the queue
RECEIVE_MESSAGE()	Display Task	Receives data from the queue
WAIT()	Both Tasks	Provides delay / blocks until event

KEY POINTS:

- Message queue enables safe data transfer between tasks.
- Sensor Task produces data and Display Task consumes it.
- SEND_MESSAGE blocks if queue is full.
- RECEIVE_MESSAGE blocks if queue is empty.
- Ensures synchronization and prevents data loss.

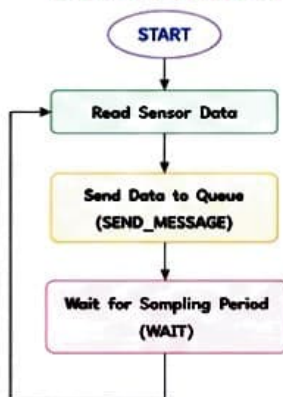
3

INITIALIZATION TASK

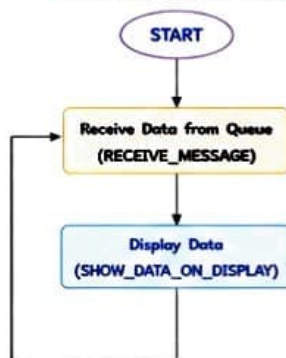


LOGICAL FLOW DIAGRAM

SENSOR TASK (PRODUCER)



DISPLAY TASK (CONSUMER)



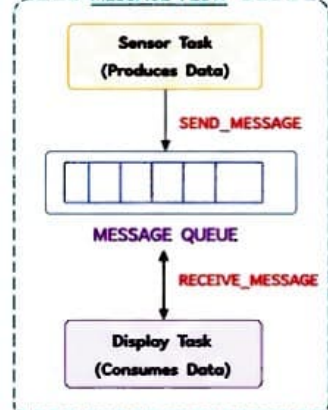
EXPLANATION:

The Sensor Task periodically reads data from a sensor and sends it to the message queue. The Display Task waits for messages in the queue, receives the data, and displays it on the screen. Message queue ensures safe and efficient communication between the two tasks.

ASSUMPTIONS:

- Both tasks are created successfully.
- Message queue is created with sufficient length.
- WAIT_FOREVER is used for blocking send/receive operations.
- Sampling period is fixed and appropriate.

MESSAGE FLOW



4